12-Year SRE / DevOps · Real-Time Troubleshooting

Production Incident Scenario Bank

Real troubleshooting scenarios across Kubernetes, Terraform, Docker, Jenkins, GitHub Actions, ArgoCD, Prometheus, Grafana, AWS, Azure, general production incidents, and AVP/Lead-level judgment calls. Each card: symptoms → diagnosis path → root cause → fix → prevention.

Coverage SLO

63% confident (48/76)

Green = confident · Amber = flagged for revision.

Search scenarios… (e.g. CrashLoopBackOff, state lock, 502, OOMKilled)

Expand all

Collapse all

All

Kubernetes

Terraform

Docker

Jenkins

GitHub Actions

ArgoCD

Prometheus

Grafana

AWS

Azure

Production Issues

AVP / Lead Level

76 of 76 scenarios

Symptoms

New pods restart every few seconds; old pods (if maxUnavailable allows) still serving traffic; rollout appears stuck.

Diagnosis

kubectl get pods -o wide

kubectl describe pod <pod> # Events + last exit code

kubectl logs <pod> --previous # logs from the crashed instance

Read the exit code in describe: 1 = app error, 137 = OOMKilled, 0-but-looping = wrong entrypoint/command.

Check what actually changed: kubectl rollout history deployment/app and diff the image tag / env / ConfigMap against the last good revision.

Check CreateContainerConfigError separately — that's a missing ConfigMap/Secret key, not a crash.

Typical root cause

New image references an env var or config key that doesn't exist yet, a DB migration wasn't run before the app started, or a new liveness probe is too aggressive for a slower cold start.

Fix

kubectl rollout undo deployment/app to restore service immediately, then fix forward — add the missing config, adjust startupProbe, or run the migration — before re-rolling.

✓ Confident

⚠ Needs revision

Symptoms

kubectl get pods shows Pending indefinitely; no container has started.

Diagnosis

kubectl describe pod <pod> # Events section says exactly why

'0/12 nodes available: insufficient cpu' → cluster is full or requests are oversized for what's left.

'node(s) didn't match Pod's node affinity/taints' → selector/taint mismatch.

PVC unbound → check kubectl get pvc; StorageClass missing or zone mismatch.

ResourceQuota in the namespace blocking creation.

Typical root cause

Cluster Autoscaler is configured but hasn't added a node yet (check its logs / max node cap), or requests were copy-pasted from a bigger workload and no single node has that much room.

Fix

Right-size the pod's requests, raise the ASG/node pool max if capacity-bound, or fix the PVC/StorageClass. Confirm Cluster Autoscaler logs show it noticing the pending pod at all — if not, check its RBAC or that the node pool is in its managed list.

✓ Confident

⚠ Needs revision

Symptoms

kubectl get nodes shows NotReady; pods on that node go into Terminating/Unknown; workloads reschedule elsewhere causing a capacity crunch.

Diagnosis

kubectl describe node <node> # Conditions section

kubectl get events --field-selector involvedObject.name=<node>

then SSH/SSM to the node itself:

systemctl status kubelet

journalctl -u kubelet -e

Conditions to check: MemoryPressure, DiskPressure, PIDPressure, Ready=Unknown (means the API server hasn't heard a heartbeat).

On the node: is kubelet even running? Disk full (df -h)? Network path to the API server broken (SG/NSG change)?

Typical root cause

Disk pressure from unbounded container logs/images filling the node, or a network change (SG/NSG/route table) silently cut kubelet's path to the control plane.

Fix

Cordon and drain the node (kubectl cordon + drain) to stop new scheduling, free disk or restore network path, then uncordon. If unrecoverable, let Cluster Autoscaler/ASG replace it — never leave a flapping node in the pool.

✓ Confident

⚠ Needs revision

Symptoms

~5-10% of requests through the Service fail; direct pod curls succeed; error rate correlates with deploys or pod restarts.

Diagnosis

kubectl get endpoints <svc> # is the failing pod even listed?

kubectl get pods -o wide --show-labels

kubectl describe pod <pod> # readiness probe status

If a pod appears in Endpoints while still warming up, traffic hits it before it's actually ready — classic missing/weak readiness probe.

If a pod stays in Endpoints briefly after being killed (during rolling update), that's a missing preStop hook / termination grace period issue.

Typical root cause

No readiness probe (or one that returns 200 before dependencies are warm), so kube-proxy sends traffic to a pod that isn't ready to serve; or on shutdown, the pod stops accepting connections before it's removed from Endpoints.

Fix

Add/tighten a readiness probe tied to real dependency health, add a short preStop sleep so the pod keeps serving while it's being deregistered from Endpoints, and set terminationGracePeriodSeconds to cover in-flight requests.

✓ Confident

⚠ Needs revision

Symptoms

Pod restarts with exit 137; kubectl describe pod shows Reason: OOMKilled; app's own APM memory metric looks well under the limit.

Diagnosis

kubectl top pod <pod>

kubectl describe pod <pod> | grep -A3 Limits

Compare the container's actual RSS (cgroup memory, what kubectl top

Check for a sidecar (Envoy/Istio, logging agent) sharing the pod that's actually the one growing.

Typical root cause

The app's memory metric only reports heap, not total process RSS — off-heap/native memory growth (e.g. connection pools, native libs, or a leak in a non-heap cache) is invisible to the app dashboard but very visible to the kernel's OOM killer.

Fix

Raise the limit to a realistic ceiling with headroom as a stopgap, but instrument actual container RSS (cAdvisor/Prometheus container_memory_working_set_bytes) as the source of truth going forward, not the app's self-reported number.

✓ Confident

⚠ Needs revision

Symptoms

Pod stuck, describe shows Failed to pull image or ImagePullBackOff.

Diagnosis

kubectl describe pod <pod> # exact error string in Events

'manifest unknown' / 'not found' → wrong tag, or the CI push actually failed silently.

'unauthorized' / 'authentication required' → imagePullSecret missing/expired, or node's IAM/managed-identity role lost registry pull permission.

'i/o timeout' → network path from node to registry blocked (private registry + NSG/SG change, or no NAT route from a private subnet).

Typical root cause

Most common in practice: the CI pipeline tagged and pushed to a registry the cluster doesn't have a valid pull credential for (rotated secret, new registry, or a node IAM role that lost ECR/ACR permissions in a recent Terraform change).

Fix

Verify the tag exists in the registry directly (docker pull or registry UI), recreate/rotate the imagePullSecret or fix the node's registry-pull role, and add a CI step that fails the pipeline if the push itself didn't succeed rather than assuming it did.

✓ Confident

⚠ Needs revision

Symptoms

Pod stays Pending; kubectl get pvc shows Pending status indefinitely.

Diagnosis

kubectl describe pvc <name> # Events show the provisioner's error

No StorageClass specified and no default one set in the cluster.

Provisioner (EBS/Azure Disk CSI) failing — check its controller pod logs in kube-system.

Requested access mode (RWX) not supported by the underlying disk type (block storage is RWO-only).

volumeBindingMode: WaitForFirstConsumer — PVC won't bind until a pod is scheduled, which is normal, not a bug.

Typical root cause

Either no default StorageClass exists in a newly built cluster, or an app asked for RWX (shared read-write) on a StorageClass backed by block storage which only supports RWO.

Fix

Set/point to a correct StorageClass, or switch to a file-based backend (EFS/Azure Files CSI) if RWX is genuinely required.

✓ Confident

⚠ Needs revision

Symptoms

Intermittent could not resolve host errors inside pods; external DNS lookups slow or fail under load; internal svc.cluster.local lookups mostly fine.

Diagnosis

kubectl get pods -n kube-system -l k8s-app=kube-dns

kubectl logs -n kube-system <coredns-pod>

kubectl top pod -n kube-system -l k8s-app=kube-dns

CoreDNS pods CPU-throttled or too few replicas for cluster size.

Check ndots:5 in the pod's resolv.conf — every external lookup tries 5 useless internal suffixes first, multiplying DNS load.

conntrack table exhaustion on the node (common at scale) causing dropped UDP DNS packets.

Typical root cause

ndots:5 amplification: an app doing many external API calls generates 5x the DNS queries it needs, overwhelming CoreDNS or the node's conntrack table under load.

Fix

Scale CoreDNS replicas / enable autoscaling for it, add a trailing dot or set dnsConfig.ndots: 1 for external-heavy workloads, and raise

net.netfilter.nf_conntrack_max on nodes if conntrack drops are visible.

✓ Confident

⚠ Needs revision

Symptoms

kubectl rollout status hangs; mix of old and new ReplicaSet pods running well past the deploy window; old pods stuck Terminating.

Diagnosis

kubectl get pods # look for long-Terminating pods

kubectl describe pod <stuck-pod> # finalizers? preStop hook hanging?

Check for a preStop hook or graceful shutdown logic that's hanging (e.g. waiting on a connection drain that never completes).

Check for a stuck finalizer (common with some storage/CSI or service-mesh sidecar injectors).

PodDisruptionBudget blocking eviction because too few pods are currently Ready.

Typical root cause

A PDB with minAvailable set too strictly relative to the new pods' actual readiness, combined with new pods themselves being slow to pass readiness — the rollout can't evict old pods without violating the PDB, and can't finish because new pods aren't Ready yet. A self-inflicted deadlock.

Fix

Diagnose why new pods aren't passing readiness first (that's the real problem); temporarily loosen the PDB or force-delete a stuck pod only as a last resort, since it bypasses the safety the PDB exists to provide.

✓ Confident

⚠ Needs revision

Symptoms

Pending pods citing insufficient resources; node count unchanged for many minutes; CA is definitely installed and running.

Diagnosis

kubectl logs -n kube-system deployment/cluster-autoscaler | tail -100

Look for 'MaxNodesTotal reached' — the node pool is already at its configured max.

'pod didn't trigger scale-up' with a reason — often a node affinity/taint the CA can't satisfy with any available instance type, or a PVC tied to a specific zone with no capacity there.

Confirm the pending pod's namespace/node pool is actually in scope for that CA instance (relevant in multi-node-group setups).

Typical root cause

The node pool's max size was left at its original sizing and nobody raised it as the workload grew, or the pod's PVC is zone-pinned to an AZ where that node pool has no room and CA correctly refuses to add a node that can't satisfy the constraint.

Fix

Raise the node pool's max node count, or fix the zone mismatch between the PVC and the node pool's available AZs.

✓ Confident

⚠ Needs revision

Symptoms

Every kubectl command is sluggish; deploys stall; even kubectl get pods takes 10+ seconds.

Diagnosis

kubectl get --raw /healthz

kubectl get componentstatuses # (deprecated but often still useful)

check etcd directly if you have access:

ETCDCTL_API=3 etcdctl endpoint health -w table

API server CPU/memory saturation — check its pod metrics.

etcd disk latency (etcd is extremely sensitive to disk fsync latency — check underlying storage IOPS).

A controller or operator hammering the API server with excessive LIST/WATCH calls (a common self-inflicted cause after adding a new controller).

Typical root cause

etcd running on storage that can't sustain its required fsync latency (common on undersized or shared-tenant disks), or a misbehaving controller doing unpaginated LIST calls on a large resource type repeatedly.

Fix

Move etcd to dedicated low-latency SSD storage, and audit recently added controllers/operators for excessive API calls via apiserver_request_total metrics broken down by client.

✓ Confident

⚠ Needs revision

Symptoms

kubectl get hpa shows current CPU above target but replica count unchanged; app is visibly struggling.

Diagnosis

kubectl describe hpa <name> # Events + 'unable to get metrics' errors

Metrics-server not running/healthy → HPA has no data at all.

Pods missing CPU requests → HPA math (% of request) has no denominator, HPA effectively can't compute anything.

Already at maxReplicas — check the ceiling, not just current vs desired.

Stabilization window delaying scale-up right after a scale-down event.

Typical root cause

Missing CPU requests on the pod spec — HPA on CPU is fundamentally a percentage of the request, and without a request set, the controller has nothing to divide by.

Fix

Set resource requests on the deployment, confirm metrics-server is healthy, and raise maxReplicas if the ceiling was actually the limiter.

✓ Confident

⚠ Needs revision

Symptoms

Apply refuses to run, error names a Lock ID, owner, and 'created' timestamp.

Diagnosis

Check the timestamp and owner in the error — is this a lock from an apply that's genuinely still running (another engineer, or a CI job), or one that crashed/was killed without releasing it?

Confirm in CI/CD history whether the referenced pipeline run is still active.

Typical root cause

A previous apply was killed (CI timeout, laptop closed mid-apply, network drop) before it could release the lock — the lock itself (DynamoDB row / Azure blob lease) survives even though the process is gone.

Fix

Confirm no apply is genuinely in progress, then terraform force-unlock <LOCK_ID>. Never force-unlock while someone might actually be applying — that's how state gets corrupted. Prevention: run all applies through CI with a hard timeout so a hung job can't hold a lock indefinitely.

✓ Confident

⚠ Needs revision

Symptoms

terraform plan shows unexpected -/+ lines on resources with no related code change.

Diagnosis

terraform plan -out=tfplan

terraform show -json tfplan | jq '.resource_changes[] | select(.change.actions==["delete","create"])'

Check if a module version bump changed a resource's internal implementation (e.g. a provider upgrade changed how a resource is named/tagged, forcing recreation).

Check for drift — did someone change this resource manually in the console? terraform plan is comparing against real infra, not just your diff.

Look for a changed argument that's ForceNew in the provider schema — some attributes can't be updated in place and always mean destroy+create.

Typical root cause

An immutable/ForceNew attribute was touched incidentally by a refactor (e.g. renaming a variable that feeds into a resource's name argument), or a provider version bump changed default behavior.

Fix

Read the plan's replacement reason carefully (Terraform 1.x shows 'must be replaced because...'), revert the incidental change if unintended, or if the recreation is real and necessary, sequence it deliberately (create-before-destroy, or during a maintenance window) rather than letting CI apply it blind.

✓ Confident

⚠ Needs revision

Symptoms

Plan wants to recreate a resource that already exists; or terraform state list is missing something real; or state file won't parse.

Diagnosis

Pull a fresh copy of remote state and inspect: terraform state pull > backup.tfstate.

Check backend versioning (S3 versioning / Azure blob soft-delete) for a prior good version to compare against.

Did two applies run concurrently despite locking (e.g. locking misconfigured, or someone bypassed it with -lock=false)?

Typical root cause

Concurrent applies without proper locking corrupting the write, or someone manually edited the state file by hand and introduced invalid JSON/drift.

Fix

Restore the last known-good version from backend versioning, then reconcile: terraform import anything genuinely missing, terraform state rm anything that shouldn't be tracked. Always state pull a backup before any manual surgery. Prevention: enforce locking is never bypassable, applies only run through CI.

✓ Confident

⚠ Needs revision

Symptoms

Apply sits at 'Still creating... [Xm elapsed]' far past normal duration for that resource type.

Diagnosis

Enable verbose logs: TF_LOG=DEBUG terraform apply to see the actual API call it's stuck on.

Check the cloud provider's own console/activity log for that resource — is the underlying operation itself stuck (e.g. an RDS instance stuck 'modifying', an AKS cluster stuck 'updating')?

Check for a dependent resource with a slow health check / wait condition (e.g. waiting for an ASG to reach healthy count that will never be met).

Typical root cause

The resource has a built-in wait condition (health check, readiness gate) that will never succeed because of an unrelated misconfiguration — Terraform isn't actually hung, it's correctly waiting on something broken.

Fix

Find and fix the underlying blocker (usually visible in the cloud console before it's visible in Terraform's log), let the apply either complete or time out cleanly, don't Ctrl+C an apply mid-flight — that's a common cause of the state corruption scenario above.

✓ Confident

⚠ Needs revision

Symptoms

Nobody touched dev's config, but the next scheduled/triggered apply in dev fails or changes unexpected things.

Diagnosis

Check the module source reference — is it pinned to a tag (?ref=v1.4.0) or floating (?ref=main/unpinned)?

Diff the module's Git history between the last known-good commit and HEAD.

Typical root cause

The module reference was unpinned (or pinned to a branch, not a tag), so a change merged for one team's use case silently propagated to every consumer on their next plan/apply.

Fix

Pin every module reference to an immutable version tag, never a branch. Roll the environments forward or back to a known-good tag explicitly, and treat this as a policy gap — add a CI check that rejects unpinned module sources.

✓ Confident

⚠ Needs revision

Symptoms

Security review or an engineer greps the state file and finds a plaintext secret.

Diagnosis

Confirm scope: is the state file itself stored securely (encrypted backend, access-controlled), or was it also exposed further (CI logs, a shared drive, a public repo)?

Typical root cause

Terraform state always contains resource attribute values in plaintext by design — marking a variable sensitive = true only redacts CLI/plan output, it does not encrypt the state file itself. Someone assumed sensitive was sufficient protection.

Fix

Rotate the exposed credential immediately, regardless of where else it's been seen. Ensure the backend itself is encrypted at rest (S3 SSE / Azure Storage encryption) and access-restricted via IAM/RBAC — that's the real control, not the sensitive flag. Prefer resources that support write-only/external secret references (secret pulled from Vault/Key Vault at apply time) over ones that store the secret's value directly in state.

✓ Confident

⚠ Needs revision

Symptoms

docker ps -a shows Exited; container never stays up.

Diagnosis

docker logs <container>

docker inspect <container> --format='{{.State.ExitCode}} {{.State.OOMKilled}}'

Exit 0 = process finished normally — usually means the entrypoint isn't actually a long-running process (a script that runs and exits, or CMD misconfigured).

Exit 1 (or other nonzero) = app-level error — read the logs, it's almost always right there.

Exit 137 = OOMKilled or SIGKILL — check OOMKilled: true specifically, and the container's memory limit.

Typical root cause

Most common: shell-form CMD/ENTRYPOINT wraps the process in /bin/sh -c, so the intended long-running process either isn't actually PID 1 or a background/exec bug causes the shell to exit immediately after launching it.

Fix

Use exec-form ENTRYPOINT ["executable","arg"], confirm the intended process actually blocks/foregrounds, and check logs for the specific app error on nonzero exits.

✓ Confident

⚠ Needs revision

Symptoms

Every build re-runs apt-get install/npm install from scratch even when dependencies haven't changed.

Diagnosis

Check the Dockerfile's instruction order — is COPY . . happening before the dependency-install step?

Check if CI is building without a cache mount/registry cache at all (fresh runner every time = no local layer cache to reuse).

Typical root cause

Copying the entire source tree before installing dependencies means any source change invalidates the cache for every layer after it, including the expensive install step — cache invalidation cascades from the wrong instruction order.

Fix

Reorder: copy only dependency manifests (package.json/go.mod) first, run install, then copy the rest of the source. In CI, use BuildKit's --cache-from/registry cache or GitHub Actions' cache action so ephemeral runners can still reuse layers across builds.

✓ Confident

⚠ Needs revision

Symptoms

curl http://app:8080 from another container fails with 'could not resolve host'.

Diagnosis

docker network ls

docker network inspect <network> # are both containers actually attached?

Are both containers on the same user-defined bridge network? The default bridge network does not provide automatic DNS resolution by container name — only user-defined networks do.

Confirm the target container is actually running and its name/alias matches what's being queried.

Typical root cause

Containers were started with docker run without --network, landing them both on the default bridge network, which has no built-in service discovery — a very common gap between 'works in docker-compose' and 'works with plain docker run'.

Fix

Create and use a user-defined bridge network (docker network create app-net, then --network app-net on both containers), or use Docker Compose which creates one automatically per project.

✓ Confident

⚠ Needs revision

Symptoms

df -h shows the disk nearly full; new containers fail to start with 'no space left on device'.

Diagnosis

docker system df -v # breakdown: images, containers, volumes, cache

docker images --filter "dangling=true"

docker ps -a --filter "status=exited"

Dangling/untagged images from repeated builds without cleanup.

Stopped containers still holding their writable layer.

Build cache growing unbounded over time.

Typical root cause

CI runners building images continuously with no cleanup policy — every build leaves behind intermediate layers and the previous image generation, and nobody's pruning.

Fix

docker system prune -af --volumes to reclaim space immediately (careful — this removes anything not actively running), then add a scheduled prune job or CI post-step, and set log rotation (--log-opt max-size) so container logs don't silently eat the rest.

✓ Confident

⚠ Needs revision

Symptoms

App inside the container is slow/thrashing; host-level memory looks fine; container has no explicit memory limit set.

Diagnosis

docker inspect <container> --format='{{.HostConfig.Memory}}'

docker stats <container>

If Memory is 0, no limit was ever set — the container can consume host memory until the host itself hits pressure, at which point the kernel's OOM killer picks a victim somewhat arbitrarily (not necessarily the offending container).

Typical root cause

Missing --memory/mem_limit on the container — without cgroup limits, a leak or an unbounded cache inside one container degrades every other container sharing that host, and the eventual OOM kill can land on an innocent neighbor.

Fix

Always set explicit memory limits per container in production, and alert on host-level MemoryPressure/swap usage, not just per-container app metrics.

✓ Confident

⚠ Needs revision

Symptoms

App logs permission errors on file writes; works fine when the container is run as root.

Diagnosis

Check the container's USER directive — running as non-root (correct security practice) means the process's UID must actually have write permission on the mounted host path/volume.

docker inspect the volume's ownership; compare to the container's runtime UID (id inside the container).

Typical root cause

The image was hardened to run as a non-root user (good practice) but the host-mounted volume's UID/GID ownership was never aligned to match, so the non-root user has no write permission on it.

Fix

chown the host path to match the container's UID, or set the container's UID to match the host path's owner explicitly, or use a named Docker volume (Docker manages permissions) instead of a raw bind mount where ownership has to be managed by hand.

✓ Confident

⚠ Needs revision

Symptoms

Build sits in the queue; Jenkins UI shows 'Waiting for next available executor'.

Diagnosis

Check node/agent labels — does any online agent actually match the pipeline's requested label?

Check if all matching agents are already saturated (all executors busy) — a queue backlog from earlier builds.

Check if an agent is offline/disconnected but still 'configured' — Jenkins won't route to it.

Typical root cause

A label typo (pipeline asks for linux-docker, agents are labeled linux_docker) or a genuinely offline/crashed dynamic agent pool (e.g. Kubernetes plugin's pod template failing to spin up new agent pods).

Fix

Fix the label mismatch, or check the agent provisioning path (K8s plugin logs, cloud agent controller) for why new agents aren't coming online. Add executor/queue-depth alerting so this is caught before someone notices a stuck build.

✓ Confident

⚠ Needs revision

Symptoms

Same pipeline succeeds on some runs, fails on others, always tracing back to the same agent node.

Diagnosis

on the agent

df -h

du -sh /var/lib/jenkins/workspace/* 2>/dev/null | sort -h | tail

Workspace directories from old builds never cleaned up.

Docker layer buildup on that specific agent if it's used for container builds.

Typical root cause

No workspace cleanup policy (ws-cleanup plugin / cleanWs() step) combined with agents that have run hundreds of builds since provisioning — disk usage grows monotonically until it tips over.

Fix

Add cleanWs() at the end of pipelines, schedule periodic disk cleanup on static agents, or move to ephemeral pod-based agents (Kubernetes plugin) that are destroyed after every build so there's nothing to accumulate.

✓ Confident

⚠ Needs revision

Symptoms

Auth failure only in specific pipelines/folders despite referencing the same Jenkins credential ID.

Diagnosis

Check the credential's scope — Jenkins credentials can be scoped to a specific folder/job; a pipeline outside that scope simply can't see it, and the failure often looks like a bad credential rather than a scope mismatch.

Confirm the failing pipeline is using withCredentials correctly and not accidentally referencing a same-named credential defined at a different scope level (folder vs global) with stale/different values.

Typical root cause

Credential scoping — the same ID exists at two different scope levels (e.g. a global one and a folder-level one with the same name), and pipelines pick up whichever is visible to them, which isn't always the one the engineer thinks.

Fix

Standardize on one scope level per credential purpose, avoid duplicate IDs across scopes, and verify with withCredentials debug output (careful not to print the secret itself) which credential actually resolved.

✓ Confident

⚠ Needs revision

Symptoms

Sudden, simultaneous failures across many unrelated pipelines right after a shared-library commit.

Diagnosis

Check the library's Git log for the recent commit and what changed in vars/ or src/.

Check whether pipelines reference the library unpinned (@Library('mylib') _ with no version) or pinned to a floating branch.

Typical root cause

Pipelines loaded the shared library unpinned (defaulting to the latest commit on the default branch), so a breaking change to a shared step landed on every consumer simultaneously with zero rollout control.

Fix

Revert the breaking library commit to restore everyone immediately, then require every pipeline to pin an explicit library version (@Library('mylib@v2.3.0')), and treat the library itself like a versioned dependency with a changelog and semver discipline.

✓ Confident

⚠ Needs revision

Symptoms

UI hangs or 502s; existing builds may still be running but nothing new can be triggered; eventually the process restarts or has to be manually restarted.

Diagnosis

on the master

jstat -gcutil <pid> 1000

java -jar jenkins-cli.jar ... # or check GC logs directly

top; free -h

Java heap exhaustion — check GC logs for long/frequent full-GC pauses right before the hang.

Too many pipelines running heavy Groovy logic (not properly using node/agent blocks) directly on the master's executor.

Plugin memory leak after a recent plugin upgrade.

Typical root cause

Pipelines doing real work directly on the master's lightweight executors instead of delegating to agents — the master is meant to orchestrate, not execute, and CPU/memory-heavy steps running there starve the whole instance.

Fix

Audit pipelines for work happening outside a proper agent block, increase master heap as an immediate mitigation, and move toward ephemeral Kubernetes agents so the master stays a thin orchestrator. Disable the built-in master executors entirely once agents are reliable (a well-known Jenkins hardening step).

✓ Confident

⚠ Needs revision

Symptoms

No workflow run appears in the Actions tab after a push that should have triggered it.

Diagnosis

Check the on: trigger block — branch filters (branches: [main]) are exact-match; a push to a differently-named default branch won't match.

Check paths: filters — if the changed files don't match, the workflow correctly won't run.

Confirm the workflow YAML itself is valid — a syntax error means GitHub silently won't register the workflow at all, with no error surfaced anywhere obvious.

Check repository Actions settings — are Actions disabled for this repo/org, or restricted to specific workflows?

Typical root cause

Most often a YAML syntax error introduced in the same commit that was supposed to add/modify the workflow — GitHub just doesn't show the workflow as available, which looks identical to 'didn't trigger'.

Fix

Validate the workflow YAML locally (actionlint or GitHub's own workflow editor which flags syntax errors), check the Actions tab's 'All workflows' list to confirm it's registered at all, and re-check trigger filters against the actual branch/path that changed.

✓ Confident

⚠ Needs revision

Symptoms

secrets.MY_TOKEN resolves empty specifically on PRs from forks, but works fine on PRs within the same repo.

Diagnosis

Confirm the trigger is pull_request (not pull_request_target) — this is expected, deliberate GitHub behavior, not a bug.

Typical root cause

This is a security feature, not a misconfiguration: pull_request-triggered workflows from forks never receive repository secrets, by design, because a malicious fork PR could otherwise exfiltrate them.

Fix

If secrets are genuinely required (e.g. to run a deploy preview), split the workflow: use pull_request_target combined with strict path/permission controls and a required-reviewer gate, and never check out the fork's untrusted code with elevated permissions in that context — that combination is the classic supply-chain vulnerability pattern GitHub explicitly warns about.

✓ Confident

⚠ Needs revision

Symptoms

Workflow run stuck 'Waiting for a runner to pick up this job'; runner shows offline in repo/org settings.

Diagnosis

on the runner host

systemctl status actions.runner.*

journalctl -u actions.runner.* -e

Runner process crashed or the host itself is unreachable (check basic connectivity/health first).

Runner's registration token expired if it wasn't set up as a persistent service.

Network egress to GitHub's Actions endpoints blocked by a recent firewall/proxy change.

Typical root cause

The runner service crashed (OOM, disk full from build artifacts) and, if not managed by a supervisor (systemd with restart policy, or an ephemeral autoscaling pool), never came back on its own.

Fix

Restart the runner service, fix the underlying resource exhaustion, and move to an autoscaling ephemeral runner setup (e.g. actions-runner-controller on Kubernetes) so individual host failures don't create a queue backlog.

✓ Confident

⚠ Needs revision

Symptoms

Same workflow, same commit — ubuntu-latest / node 18 passes, windows-latest / node 18 fails with a path or line-ending error.

Diagnosis

Check for hardcoded Unix-style paths (/ vs \) in scripts that run cross-platform.

Check for line-ending sensitivity (CRLF vs LF) if the failure involves shell scripts or diffing generated files.

Compare exact tool/dependency versions resolved per OS — 'latest' can silently drift between runner images.

Typical root cause

Platform-specific path or shell assumptions baked into a script that was only ever tested on one OS — the matrix is doing exactly its job of surfacing that gap.

Fix

Use cross-platform path handling (language-native path libraries instead of string concatenation), pin exact tool versions rather than 'latest' per OS, and set .gitattributes to normalize line endings if that's the culprit.

✓ Confident

⚠ Needs revision

Symptoms

Billing alert or usage dashboard shows a sudden jump in Actions minutes consumed.

Diagnosis

Check for a workflow stuck in a retry/re-trigger loop (e.g. a workflow that pushes a commit, which re-triggers itself on push).

Check for a scheduled (cron) workflow that got more frequent than intended, or one left running on every branch instead of just main.

Check concurrent job count — a matrix with a much larger fan-out than intended (e.g. an unbounded matrix generated from dynamic input) multiplies minutes fast.

Typical root cause

A workflow that commits back to the repo (auto-formatting, version bumping) without excluding its own commits from re-triggering itself — a classic infinite-loop-of-workflow-runs pattern.

Fix

Add [skip ci] to bot commits or use a dedicated trigger condition excluding the bot's own pushes, add concurrency groups to cancel superseded runs, and set a sane timeout-minutes on every job so a hang doesn't burn minutes indefinitely.

✓ Confident

⚠ Needs revision

Symptoms

ArgoCD UI shows OutOfSync; clicking Sync either fails or completes but the app immediately shows OutOfSync again.

Diagnosis

argocd app get <app> --hard-refresh

argocd app diff <app>

app diff shows exactly what's different between Git and the live cluster state — read it, don't guess.

Check for a mutating admission webhook or controller (e.g. an operator adding default annotations) modifying the resource after ArgoCD applies it — ArgoCD sees that as drift and flips back to OutOfSync.

Typical root cause

Something outside ArgoCD's control is mutating the live resource after sync — a defaulting webhook, an HPA changing replica count that's also set in Git, or a service mesh sidecar injector adding fields ArgoCD doesn't expect — creating a permanent tug-of-war.

Fix

Identify the mutator and either exclude the specific field from ArgoCD's diff via ignoreDifferences in the Application spec (e.g. ignore spec.replicas if an HPA owns it), or stop managing that field in Git at all if a controller is meant to own it.

✓ Confident

⚠ Needs revision

Symptoms

Green 'Healthy' status in ArgoCD UI, but kubectl get pods for that app shows CrashLoopBackOff.

Diagnosis

ArgoCD's health check for a Deployment is based on the rollout status conditions

Check if a custom health check (Lua script) is configured for this resource type and whether it's actually correct.

Typical root cause

Default health assessment for the resource type doesn't match the real failure mode — this is especially common with CRDs/custom resources that have no built-in ArgoCD health logic and report Healthy simply because ArgoCD doesn't know how to evaluate them at all.

Fix

Don't rely on ArgoCD's health status as your only signal — pair it with actual Prometheus/Kubernetes-level alerting on pod restarts and readiness. For CRDs, write a custom Lua health check so ArgoCD's status reflects reality.

✓ Confident

⚠ Needs revision

Symptoms

Sync operation fails outright rather than the app just drifting; error references a specific manifest and a Kubernetes API validation error.

Diagnosis

argocd app sync <app> --dry-run

kubectl apply --dry-run=server -f <manifest> # reproduce outside ArgoCD

Reproduce the exact validation error with plain kubectl — it's almost always a real schema issue (immutable field change, invalid enum value, missing required field) that would fail even applied manually.

Check if a CRD version mismatch is the cause — the manifest targets a CRD version/schema that isn't installed in this cluster yet.

Typical root cause

An attempt to change an immutable field in place (e.g. a Service's clusterIP, or a StatefulSet's volumeClaimTemplates) — Kubernetes rejects the update regardless of who's applying it, ArgoCD included.

Fix

For a genuinely immutable field change, the resource needs to be deleted and recreated deliberately (often via a PreSync hook or a manual out-of-band step), not just re-synced — communicate that clearly since it may cause brief downtime for that resource.

✓ Confident

⚠ Needs revision

Symptoms

App shows 'Unknown' status; repo connection test in ArgoCD Settings fails.

Diagnosis

argocd repo list

argocd repo get <repo-url>

Check whether the credential (SSH key / PAT) stored in ArgoCD's repo secret has expired or been rotated on the Git provider's side without updating ArgoCD.

Check network path — if the Git server is internal/on-prem, confirm ArgoCD's pod can actually route to it (NetworkPolicy, firewall).

Typical root cause

A PAT or deploy key rotated/expired on the Git provider side (common with a security policy enforcing periodic token rotation) without a corresponding update to the ArgoCD repo-credentials secret — a silent breakage waiting to happen on the token's expiry date.

Fix

Rotate and update the credential in ArgoCD's repo secret, and prefer a GitHub/GitLab App or short-lived federated credential over a long-lived PAT so rotation doesn't require manual coordination going forward.

✓ Confident

⚠ Needs revision

Symptoms

App cycles between Synced and OutOfSync repeatedly; replica count or another field keeps bouncing between two values.

Diagnosis

Identify the field that's bouncing — usually spec.replicas when both Git (via ArgoCD) and an HPA are trying to own it.

Typical root cause

Git defines a static replica count while an HPA is actively scaling the same Deployment — ArgoCD keeps resetting it to Git's value, the HPA keeps changing it back, forever.

Fix

Add ignoreDifferences for spec.replicas on that resource in the Application spec so ArgoCD stops treating HPA-driven changes as drift — this is a well-known, expected pattern, not a workaround to be embarrassed about.

✓ Confident

⚠ Needs revision

Symptoms

A resource that should exist per Git is missing from the cluster; ArgoCD still shows the app as Synced/Healthy or takes a long time to notice.

Diagnosis

Check the Application's sync policy — is selfHeal enabled? Without it, ArgoCD only reconciles on the next scheduled/triggered sync, not automatically on drift detection.

Check the polling/reconciliation interval — default is a few minutes, which can look like 'ArgoCD isn't doing anything' if you're watching right after the deletion.

Typical root cause

automated.selfHeal not enabled on the Application — ArgoCD detects drift on its next refresh but won't act on it without selfHeal turned on, which is often left off intentionally in early rollouts and then forgotten.

Fix

Enable selfHeal: true in the sync policy for apps where drift should always be corrected automatically, and separately, treat a manual kubectl delete against a GitOps-managed resource as a process gap worth addressing — access should generally be locked down so Git is the only path to change.

✓ Confident

⚠ Needs revision

Symptoms

Target shows red/down in Prometheus UI; up{job="..."} == 0; gaps appear in that target's metrics.

Diagnosis

Manually curl the target's /metrics endpoint from a pod on the same network to see if it's slow or hanging.

Check if the target's metrics endpoint has grown huge (cardinality explosion) — scraping now takes longer than the configured scrape_timeout.

Check network path — NetworkPolicy or SG change blocking Prometheus from reaching the target.

Typical root cause

A recent code change added a high-cardinality label (user ID, request ID, raw URL path) to a metric, ballooning the number of time series the target has to serialize on every scrape until it exceeds the timeout.

Fix

Identify and remove the high-cardinality label at the source (app code), or as an immediate mitigation increase scrape_timeout while the fix rolls out — but treat cardinality as the root cause to fix, not the timeout as the thing to keep raising.

✓ Confident

⚠ Needs revision

Symptoms

Prometheus pod repeatedly OOMKilled; Grafana dashboards time out; promtool/API queries take tens of seconds for what should be simple lookups.

Diagnosis

curl localhost:9090/api/v1/status/tsdb | jq .

look at: numSeries, and topN by label cardinality

Check /api/v1/status/tsdb for the highest-cardinality label names and label value counts — this points directly at the offending metric.

Check for a recent deploy that added a metric with an unbounded label (again, cardinality is the number one Prometheus failure mode).

Typical root cause

Cardinality explosion — one metric with a label like raw request path or a UUID creates millions of unique time series, each of which Prometheus must hold in memory.

Fix

Drop or relabel the offending label at scrape time (metric_relabel_configs) as an immediate fix, fix it properly at the source, and consider remote-write to Thanos/Mimir with a recording-rule-based rollup for anything that genuinely needs high cardinality long-term.

✓ Confident

⚠ Needs revision

Symptoms

Manually running the alert's PromQL expression in the UI returns data matching the condition, but Alertmanager never receives it.

Diagnosis

promtool check rules alerts.yml

in Prometheus UI: Alerts page — is it in Pending or Firing?

Check the alert's for: duration — if the condition has been true for less time than for:, it's correctly sitting in Pending, not Firing, and won't notify yet.

Check Alertmanager routing — the alert may be firing correctly but getting swallowed by an inhibition rule or routed to a receiver nobody's watching.

Check if the rules file even loaded — a YAML syntax error in one rule group can silently prevent the whole file from loading depending on config.

Typical root cause

Most often either the for: duration hasn't elapsed yet (easy to forget when testing), or an inhibition rule in Alertmanager (e.g. 'node down suppresses all pod alerts on that node') is correctly-by-design swallowing the notification.

Fix

Check the Alerts page state (Pending vs Firing) first, then trace it through Alertmanager's routing tree — amtool config routes test can simulate where a given alert's labels would route.

✓ Confident

⚠ Needs revision

Symptoms

Disk usage alert on the Prometheus volume; logs show write errors; queries against recent data fail or return incomplete results.

Diagnosis

du -sh /prometheus/*

curl localhost:9090/api/v1/status/tsdb

Check retention settings (--storage.tsdb.retention.time) against actual data volume — was retention increased without a corresponding disk size increase?

Check for the cardinality explosion pattern again — more series means more disk per unit time, not just more memory.

Typical root cause

Retention or scrape frequency was increased (more history, more targets, more frequent scrapes) without provisioning more disk to match — a capacity planning gap rather than a bug.

Fix

Reduce retention temporarily to stop the bleeding, expand the volume, and move to remote-write with a long-term store (Thanos/Mimir) so local TSDB only needs to hold a short operational window (e.g. 15 days) rather than growing indefinitely.

✓ Confident

⚠ Needs revision

Symptoms

up{job="myapp"} returns no data at all (not even 0) — the target isn't even listed anymore.

Diagnosis

Check Kubernetes service discovery relabeling — did the pod's annotations or labels used for scrape discovery (prometheus.io/scrape, or the ServiceMonitor's label selector) change or get dropped in the new manifest?

If using the Prometheus Operator, check whether the ServiceMonitor's selector still matches the Service's labels after the deploy.

Typical root cause

A manifest refactor (e.g. moving to a Helm chart, or a labeling standard change) dropped or renamed the specific label/annotation that Prometheus's service discovery relabel_configs or the ServiceMonitor selector was matching on — the target silently

stops being discovered rather than erroring.

Fix

Diff the old and new manifests' labels/annotations specifically, restore the expected scrape annotation or ServiceMonitor-matching label, and add a meta-alert ('expected job X has zero targets') so a silent discovery loss like this pages someone instead of going unnoticed.

✓ Confident

⚠ Needs revision

Symptoms

Same PromQL query run in the Prometheus UI returns results; the identical query in a Grafana panel shows No data.

Diagnosis

Check the panel's configured time range against when the data actually exists — an absolute time range left over from editing, or a dashboard-level time range override, is the single most common cause.

Check the datasource selected on that specific panel — dashboards can have panels pointing at different/stale datasource UIDs after a datasource was recreated.

Check template variable values — if a $var used in the query resolves to something that matches no series, you get No data with no obvious error.

Typical root cause

A template variable resolving to an unexpected/empty value, or a stale datasource reference left over from a datasource migration (e.g. Prometheus datasource was deleted and recreated with a new UID, but panels still reference the old one).

Fix

Inspect the panel's actual executed query (Grafana's query inspector shows the final resolved query, not just the template) to see exactly what was sent, and re-point any panel with a stale datasource UID.

✓ Confident

⚠ Needs revision

Symptoms

Dashboard spins for 30+ seconds or errors out entirely; other, simpler dashboards load fine.

Diagnosis

Check for a panel querying with too wide a time range combined with a high-cardinality query (e.g. no aggregation, grouping by a high-cardinality label like pod name across 30 days).

Check the refresh interval — an auto-refresh set too aggressively (5s) on a heavy dashboard compounds load on the datasource continuously.

Count the panels — a dashboard with 40+ panels each firing its own query on load is simply a lot of concurrent load, especially right after a browser refresh.

Typical root cause

An unaggregated query (no sum by()/avg by()) over a long time range and high-cardinality label, forcing Prometheus to return and Grafana to render far more series than the panel can usefully display.

Fix

Aggregate at the query level rather than the visualization level, cap default time ranges sensibly, and use recording rules for expensive queries that back frequently-viewed dashboards so the heavy computation happens once on a schedule, not on every page load.

✓ Confident

⚠ Needs revision

Symptoms

Alert state shows Firing/Alerting in Grafana; the configured contact point never receives anything.

Diagnosis

Check the notification policy tree — does the alert's labels actually match the route that points to the intended contact point, or does it fall through to a default route pointing somewhere else/nowhere?

Test the contact point directly (Grafana has a 'Test' button) to isolate whether it's a routing problem or a genuinely broken webhook/integration.

Check for a silence that's inadvertently still active, muting the notification even though the alert state shows Firing.

Typical root cause

A notification policy routing tree matched the alert to the wrong (or default/unconfigured) contact point — the alert engine itself is working correctly, but the policy layer between 'alert fired' and 'notification sent' has a gap.

Fix

Trace the alert's labels through the notification policy tree explicitly to see which route it actually matches, fix the routing/label mismatch, and check for stale silences regularly since they're a common source of 'the alert should have paged but didn't'.

✓ Confident

⚠ Needs revision

Symptoms

A $service or $env dropdown at the top of a dashboard shows no options, or options that no longer exist in the data.

Diagnosis

Check the variable's query (usually a label_values() call) against the current datasource — does it still return results run manually?

Check if the variable's refresh setting is 'On Dashboard Load' vs 'Never' — a variable set to never refresh keeps showing whatever values existed when it was first configured.

Typical root cause

The variable's refresh mode is set to 'Never' (or the datasource/metric it queries was renamed), so it's showing a cached list of values from whenever it was last manually refreshed, not the current state of the data.

Fix

Set the variable's refresh to 'On Dashboard Load' (or 'On Time Range Change' if values are time-dependent), and re-verify the underlying label_values() query is still valid against current metric/label names.

✓ Confident

⚠ Needs revision

Symptoms

SSH/app connections time out; instance shows Running and passes status checks in the console.

Diagnosis

Timeout (no response) vs refused (RST) tells you a lot immediately — timeout means something's dropping packets silently; refused means the path is fine but nothing's listening.

Check Security Group inbound rules for the port in question.

Check NACLs on the subnet — remember they're stateless, so both inbound AND outbound return-traffic rules matter.

Check the route table — did a recent Terraform/console change remove the IGW or NAT route?

Use VPC Reachability Analyzer to trace the exact hop where it breaks — much faster than manual tracing.

Typical root cause

A recent Terraform apply or manual console change modified a shared NACL or route table, affecting more resources than the person making the change realized — a blast-radius problem, not usually a single misconfigured instance.

Fix

Revert the specific rule/route change, confirm connectivity restored, then add the change to code review with an explicit blast-radius note next time. Reachability Analyzer or a pre-change plan diff would have caught this before it shipped.

✓ Confident

⚠ Needs revision

Symptoms

Small but consistent percentage of requests through the ALB fail with 502; direct requests to a healthy target succeed.

Diagnosis

ALB access logs -> filter on elb_status_code=502

CloudWatch: TargetResponseTime, HTTPCode_Target_5XX_Count, UnHealthyHostCount

502 specifically means the target sent a malformed/invalid HTTP response, or closed the connection while the ALB was mid-request — different from 503 (no healthy targets) or 504 (target too slow).

Check for a target-side idle timeout shorter than the ALB's, causing the target to close a kept-alive connection the ALB was about to reuse.

Check application logs on the target at the exact timestamps for a crash/restart correlating with the 502s.

Typical root cause

Keep-alive timeout mismatch — the application server's idle-connection timeout is shorter than the ALB's (default 60s), so the ALB occasionally tries to reuse a connection the backend already silently closed.

Fix

Set the target's keep-alive timeout longer than the ALB's idle timeout (or match them exactly), and separately verify the app isn't crashing/restarting at the same timestamps as a red herring.

✓ Confident

⚠ Needs revision

Symptoms

Sporadic timeouts, more frequent after periods of low invocation traffic; error correlates loosely with time-of-day.

Diagnosis

CloudWatch Logs Insights:

filter @type="REPORT" | stats avg(@duration), max(@duration) by bin(5m)

Compare cold-start-heavy periods (low traffic → more cold starts) against the timeout spikes.

If the Lambda is VPC-attached, check ENI provisioning delay specifically — VPC Lambdas historically had a much slower cold start due to ENI creation (largely mitigated post-Hyperplane, but still worth checking on older configs).

Check for a downstream dependency (RDS, an external API) that's slow to establish a fresh connection on a cold start vs reusing a warm one.

Typical root cause

Cold starts combined with a downstream connection (DB, external API) that takes meaningfully longer to establish fresh than the function's configured timeout allows,

especially if connection setup wasn't accounted for when the timeout was originally set.

Fix

Increase the timeout to cover worst-case cold-start + connection setup, use provisioned concurrency for latency-sensitive functions, and move connection setup outside the handler (module-level) so warm invocations reuse it.

✓ Confident

⚠ Needs revision

Symptoms

App logs connection errors; RDS CloudWatch shows DatabaseConnections pegged near the instance class's max.

Diagnosis

Check if a recent app deploy changed connection pool sizing, or if multiple app replicas each opening their own pool are multiplying total connections beyond what was planned.

Check for connections stuck idle/not returned to the pool (a leak in the app's connection handling).

SHOW PROCESSLIST (MySQL) or pg_stat_activity (Postgres) to see what's actually holding connections open.

Typical root cause

An autoscaling event added more app replicas, each opening its own fixed-size connection pool, and nobody recalculated total possible connections (replicas × pool size) against the DB instance's actual connection limit.

Fix

Right-size per-replica pool size against the DB's real ceiling, and put RDS Proxy in front for connection pooling/multiplexing so scaling app replicas doesn't linearly scale DB connections.

✓ Confident

⚠ Needs revision

Symptoms

403 AccessDenied on s3:GetObject; the attached IAM policy clearly has an Allow statement for that action and resource.

Diagnosis

Check the bucket policy separately — S3 access requires both the IAM policy AND the bucket policy (if one exists) to not explicitly deny; a bucket policy with a broad Deny (e.g. enforcing a VPC endpoint condition) can override an otherwise-correct IAM Allow.

Check the account's S3 Block Public Access settings and any SCP at the Organization level — an explicit Deny anywhere in the evaluation chain wins, always.

Confirm the resource ARN in the policy matches exactly — a missing /* suffix (allowing the bucket but not objects inside it) is a very common typo.

Typical root cause

An SCP or bucket policy has an explicit Deny (often for a legitimate security reason, like restricting access to a specific VPC endpoint) that the requester's IAM policy alone can never override — explicit Deny always wins over Allow, regardless of which layer it's defined in.

Fix

Use the IAM Policy Simulator or CloudTrail's actual denial reason (increasingly surfaced directly in error messages) to find exactly which layer is denying, then fix or add a legitimate exception at that layer specifically.

✓ Confident

⚠ Needs revision

Symptoms

ASG activity history shows frequent launch/terminate cycles; instance count oscillates instead of settling.

Diagnosis

Check the scaling policy's target metric and cooldown period — a metric that naturally oscillates (like CPU right at the threshold) with too short a cooldown causes rapid flip-flopping.

Check if scale-in and scale-out policies are both watching the same metric with overlapping thresholds, effectively fighting each other.

Typical root cause

Target-tracking threshold set right where normal traffic naturally hovers, combined with a cooldown period too short to let a newly launched instance actually start absorbing load before the next scaling evaluation fires.

Fix

Widen the gap between scale-out and scale-in thresholds, lengthen the cooldown to cover realistic instance warm-up time, and prefer target-tracking policies (which have built-in stabilization) over hand-rolled step-scaling for straightforward cases.

✓ Confident

⚠ Needs revision

Symptoms

AssumeRole calls that worked yesterday now fail with AccessDenied; nothing was changed on the calling side.

Diagnosis

Check the target role's trust policy — did someone edit it (even for an unrelated reason) and inadvertently narrow the trusted principal or add a condition that no longer matches?

Check for an ExternalId requirement that changed, or an SCP in the target account's Organization newly restricting sts:AssumeRole.

Check if the calling role/user's own permissions to call sts:AssumeRole on that specific target ARN were altered.

Typical root cause

The target account's role trust policy was edited as part of an unrelated security hardening pass, and the edit was broader than intended — trust policies are easy to under-test because the failure only shows up on the calling side, in a different account, often noticed by a different team.

Fix

Diff the trust policy against its last known-good version, restore the correct trusted principal/condition, and treat cross-account trust policy changes as requiring the same review rigor as any other production security-boundary change — ideally with the calling team notified before the change ships.

✓ Confident

⚠ Needs revision

Symptoms

SSH/RDP or app connections time out; VM shows Running with no reported issues.

Diagnosis

az network nsg rule list --nsg-name <nsg> --resource-group <rg> -o table

or use the portal's Network Watcher -> IP flow verify

Check both the subnet-level AND NIC-level NSG — a rule can exist on either, and effective rules are the intersection/union of both, which is easy to reason about wrong.

Use Network Watcher's IP flow verify — it tells you definitively which NSG rule (and at which level) is allowing or denying a specific flow, rather than reading rules by eye.

Check route tables (UDRs) for an unrelated change that rerouted traffic through an NVA/firewall that's now blocking it.

Typical root cause

A security-hardening NSG rule was applied at the subnet level for a different, unrelated purpose (e.g. locking down a different VM in the same subnet) and inadvertently affected every resource sharing that subnet.

Fix

Use IP flow verify to confirm the exact blocking rule, scope the NSG rule more precisely (move it to NIC level, or split the subnet), and prefer Application Security Groups over broad subnet-level CIDR rules so intent stays scoped to the actual resources involved.

✓ Confident

⚠ Needs revision

Symptoms

Pod stuck; describe shows an authentication or 'not found' error pulling from ACR specifically (other public images pull fine).

Diagnosis

kubectl describe pod <pod>

az aks check-acr --resource-group <rg> --name <cluster> --acr <acr-name>

'unauthorized' → AKS cluster's kubelet identity was never granted AcrPull on this registry, or the attachment was done manually and later drifted.

'not found' → check the tag actually exists in that specific ACR (easy to confuse dev/prod registries with similar names).

Private ACR with network restrictions → check if AKS's outbound path (or a private endpoint) can actually reach it, especially after a recent networking change.

Typical root cause

ACR attachment (az aks update --attach-acr) was done once at cluster creation but a later cluster rebuild/recreation, or a new node pool with a different identity, never got the same attachment repeated.

Fix

az aks update --resource-group <rg> --name <cluster> --attach-acr <acr-name> to (re)grant AcrPull to the cluster's identity, and add this as an explicit, codified (Terraform/Bicep) step rather than a manual one-off so it survives cluster recreation.

✓ Confident

⚠ Needs revision

Symptoms

App Gateway backend health blade shows Unhealthy; direct curl to the backend VM/pod succeeds fine.

Diagnosis

Check the health probe's configured path — it might be probing / when the app only returns 200 on a dedicated /health endpoint, or vice versa.

Check the probe's expected status codes — App Gateway defaults to expecting 200-399; an app returning a 401 on its root path (auth-protected) will show unhealthy even though it's technically 'up'.

Check NSG on the backend subnet — App Gateway's probe traffic originates from the App Gateway subnet and needs an explicit allow, separate from normal user traffic rules.

Typical root cause

The health probe path/expected-status configuration doesn't match what the backend actually serves at that path — very common when an app added authentication to its root path after the probe was originally configured against it.

Fix

Point the probe at a dedicated, unauthenticated /healthz-style endpoint that reflects real app health (including downstream dependency checks) rather than reusing a real, possibly-protected application route.

✓ Confident

⚠ Needs revision

Symptoms

App logs 403 Forbidden calling Key Vault; the resource clearly has a Managed Identity assigned.

Diagnosis

Check whether Key Vault is using access policies or the newer Azure RBAC authorization model — they're mutually exclusive per vault, and granting via the wrong mechanism silently does nothing.

Confirm the exact identity (system-assigned has a different object ID than any user-assigned identity) was granted the correct role/policy — a very common mistake is granting the wrong one when a resource has both configured.

Check Key Vault's network rules — if it's restricted to specific VNets/private endpoints, confirm the calling resource is actually inside an allowed network path.

Typical root cause

The Key Vault was recently migrated from access-policy to RBAC authorization mode (or vice versa) as part of a security hardening effort, and the corresponding role assignment for this specific identity wasn't recreated under the new model.

Fix

Confirm which authorization model the vault is actually using (az keyvault show --query properties.enableRbacAuthorization), and grant the identity access via that specific mechanism — Key Vault Secrets User RBAC role, or an access policy, not both assumed interchangeably.

✓ Confident

⚠ Needs revision

Symptoms

App sees slow/timing-out queries; Azure Monitor shows DTU (or vCore CPU) consistently near 100%.

Diagnosis

-- Query Performance Insight in the portal, or:

SELECT TOP 10 * FROM sys.dm_exec_query_stats

ORDER BY total_worker_time DESC;

Query Performance Insight surfaces the top resource-consuming queries directly — start there rather than guessing.

Check for a missing index on a query that recently started running against a much larger table (data growth, not a code change, can be the trigger).

Check the service tier/DTU sizing against actual sustained load — was this sized for launch traffic and never revisited?

Typical root cause

A query that was fine against a small table becomes expensive as data volume grows past a threshold where a missing index starts mattering — a slow-burn issue that looks like a sudden incident because it crosses a tipping point rather than a single bad deploy.

Fix

Add the missing index identified by Query Performance Insight / execution plan, and separately evaluate whether the service tier needs to scale with data growth — set up a proactive DTU/CPU trend alert so the next tipping point is caught before it pages someone.

✓ Confident

⚠ Needs revision

Symptoms

Peering status shows 'Connected' in the portal on both sides; ping/connection attempts across the peering still fail.

Diagnosis

Peering status being Connected only confirms the peering relationship exists — it says nothing about NSGs, UDRs, or overlapping address spaces, all of which independently block traffic.

Check for overlapping/duplicate CIDR ranges between the two VNets — peering with an overlap is allowed to be configured but traffic will never route correctly.

Check 'Allow forwarded traffic' and 'Allow gateway transit' settings if this peering is meant to be transitive through a hub — remember VNet peering is non-transitive by default.

Typical root cause

An assumption that peering is transitive — spoke A peers to hub, spoke B peers to hub, and someone expects spoke A to reach spoke B through the hub automatically, which doesn't happen without a route table explicitly directing traffic through an NVA/firewall in the hub.

Fix

For true hub-spoke transitivity, route spoke-to-spoke traffic through an NVA/Azure Firewall in the hub via UDRs — don't rely on peering itself to provide it. Confirm no CIDR overlap as a first, cheap check.

✓ Confident

⚠ Needs revision

Symptoms

az aks show reports provisioning state stuck at 'Upgrading' far past normal duration; some nodes on the old Kubernetes version, some on the new.

Diagnosis

az aks show --resource-group <rg> --name <cluster> --query provisioningState

kubectl get nodes -o wide # compare kubelet versions across nodes

Check for a PodDisruptionBudget that's too strict for the surge upgrade to evict pods and proceed to the next node.

Check for a pod that simply won't schedule on the new node pool (taint/toleration mismatch, or a resource request that doesn't fit the new node size if the upgrade also changed VM SKU).

Typical root cause

Same root pattern as the Kubernetes rolling-update deadlock above — a PDB with minAvailable set too strictly is preventing the upgrade's node drain from evicting enough pods to proceed, and the upgrade process just waits indefinitely rather than failing loudly.

Fix

Identify the blocking PDB (kubectl get pdb -A, check which one has zero disruptions allowed currently), temporarily relax it if safe, and let the upgrade proceed — then investigate why that workload's readiness/replica count made the PDB unsatisfiable in the first place.

✓ Confident

⚠ Needs revision

Symptoms

RSS/heap graph trends upward in a straight line with no plateau; only resolved by a restart, then the climb starts again.

Diagnosis

Confirm it's a genuine leak vs an expected cache that just hasn't hit its eviction ceiling yet — check if growth ever plateaus under steady load, even after days.

Take a heap dump / profiler snapshot at two points in time and diff object counts by type — the type growing unbounded points directly at the leaking code path.

Correlate the leak rate against traffic/feature usage — does it grow faster during a specific workflow, narrowing the search.

Typical root cause

Most commonly: an unbounded in-memory cache/map (session state, a dedup set, a listener registry) that adds entries but never removes them — functionally a cache with no eviction policy, not a classic 'forgot to free memory' bug in managed-memory languages.

Fix

Bound the offending structure with an eviction policy (LRU, TTL, max size) once identified. As an immediate mitigation, a scheduled restart buys time safely, but treat that as a stopgap with an explicit ticket, not a permanent fix.

✓ Confident

⚠ Needs revision

Symptoms

A single dependency (a DB, a third-party API) gets slow, and within minutes every service calling it is also unhealthy, then services calling those services degrade too.

Diagnosis

Trace the failure's spread direction via distributed tracing — it should show latency ballooning first at the actual slow dependency, then spreading upstream through the call graph as callers' thread pools/connection pools exhaust waiting on it.

Check for missing timeouts — a call with no timeout waits forever, holding a thread/connection hostage indefinitely rather than failing fast.

Typical root cause

No timeout (or a much-too-generous one) on calls to the slow dependency, so every caller's limited pool of threads/connections gets consumed waiting, which then makes the caller itself appear unhealthy to its own callers — the failure propagates because nothing fails fast.

Fix

Immediate: identify and isolate/scale the actual slow dependency, or shed load to it. Structural: add aggressive timeouts everywhere, circuit breakers that stop calling a dependency once it's clearly unhealthy, and bulkheads (separate connection pools per dependency) so one slow dependency can't exhaust the resources needed to serve requests that don't even touch it.

✓ Confident

⚠ Needs revision

Symptoms

Periodic, sharp DB load spikes at regular intervals (often correlating with a fixed cache TTL); between spikes, DB load is fine.

Diagnosis

Check cache key TTLs — are many keys set with the exact same fixed TTL (e.g. all set to expire exactly 1 hour after a bulk cache warm), causing simultaneous mass expiry?

Confirm the DB spike timing lines up precisely with the TTL interval — a strong signal this is the pattern rather than organic load growth.

Typical root cause

A bulk cache-warming job set every key with an identical TTL, so they all expire at the same instant, and every subsequent request for any of those keys misses the cache simultaneously and hits the database at once.

Fix

Add jitter to TTLs (e.g. base TTL ± random 10%) so expiry spreads out instead of clustering, and/or implement a 'stale-while-revalidate' pattern where one request

refreshes the cache while others keep serving the (briefly stale) cached value instead of all falling through to the DB together.

✓ Confident

⚠ Needs revision

Symptoms

Intermittent transaction failures under concurrent load with a database-level deadlock error; works fine at low traffic.

Diagnosis

Pull the deadlock graph/log from the DB (Postgres logs deadlocks by default with the involved queries; SQL Server's deadlock graph is similarly explicit) — it names the exact two queries and the lock order each held.

Compare the order in which each transaction acquires locks on the same set of tables/rows.

Typical root cause

Two different code paths update the same two tables/rows but in opposite order (transaction A locks table1 then table2; transaction B locks table2 then table1) — under enough concurrency, they eventually cross and each waits on the other, and the DB kills one to break the cycle.

Fix

Enforce a single, consistent lock acquisition order across every code path touching the same resources — the standard, durable fix for this class of bug — rather than adding retries alone (retries mask it under low load and reappear under higher load).

✓ Confident

⚠ Needs revision

Symptoms

Latency on a list endpoint scales roughly linearly with the number of items returned; a single-item endpoint using the same ORM is fast.

Diagnosis

Enable query logging/APM tracing for one slow request and count the actual number of SQL queries fired — if it's roughly N+1 (one query for the list, then one more per item for a related field), that's the signature.

Typical root cause

An ORM lazily loading a related object/field per row instead of eagerly joining or batch-loading it — extremely common when a template or serializer accesses a related field per item without the developer realizing each access triggers its own query.

Fix

Use eager loading (JOIN/include/select_related depending on the ORM) or a batch loader (DataLoader pattern) to fetch the related data in one or two queries total instead of one per item — turns an O(N) query pattern into O(1).

✓ Confident

⚠ Needs revision

Symptoms

Requests queue and eventually time out waiting for a pooled connection (DB, HTTP client pool, etc.) well before the actual downstream dependency is under real stress.

Diagnosis

Check the pool's configured max size against the number of concurrent app instances/threads that could all be requesting a connection simultaneously.

Check for connections not being released back to the pool promptly — a code path that opens a connection but doesn't return it on an error path is a very common leak source.

Typical root cause

An exception-handling path that acquires a connection, hits an error, and returns/throws without releasing the connection back to the pool — under normal operation this is invisible, but under any elevated error rate it drains the pool fast.

Fix

Ensure connection release happens in a finally/using-block/context-manager so it's guaranteed regardless of the exit path, and separately size the pool against realistic peak concurrency, not average load.

✓ Confident

⚠ Needs revision

Symptoms

No alerts fired, no exceptions logged, but specific records are showing incorrect values; only noticed via customer reports or a downstream reconciliation job.

Diagnosis

This is the hardest class of incident because nothing 'failed' — start by bounding the blast radius: which records, since exactly when (correlate with deploy timestamps), and is it still actively happening or a one-time write?

Diff the new code's data-writing logic against the previous version specifically for the field(s) affected — a subtly wrong default, a unit conversion, an off-by-one in a batch job, or a schema migration that didn't backfill correctly are the classic causes.

Typical root cause

A schema/logic change shipped with a subtly wrong default value or unit assumption that produces valid-looking but incorrect data — it passes all tests because the tests didn't cover the specific edge case, and it never throws because nothing about it is technically an error.

Fix

Stop the bleeding first (roll back or feature-flag off the change), then scope and run a data-correction/backfill job for affected records, and treat this as a process gap worth a

genuine postmortem — since no monitoring caught it, ask what invariant-based check (not just error-rate monitoring) would have.

✓ Confident

⚠ Needs revision

Scenario

Payments, checkout, and notifications all fire Sev-2 alerts within two minutes of each other. Multiple engineers are already independently investigating each in isolation.

Leadership approach

First move: check for a common upstream cause before treating these as three incidents — a shared dependency (DNS, a platform-level deploy, a network path, an auth service) failing looks exactly like 'everything broke at once'.

Consolidate onto one bridge initially rather than three separate ones — splitting too early means duplicated effort and no one sees the pattern.

Assign one person specifically to check shared infrastructure/dependencies while others check each service's own logs — parallelize the hypothesis search, not just the fix.

Declare severity based on the worst-affected service (payments) and communicate accordingly, even while cause is still unclear.

What this is really testing

Whether you reflexively split into silos under pressure, or recognize the pattern of a shared root cause and coordinate the investigation accordingly — the second instinct is what separates a senior IC from someone who just triages loudly.

✓ Confident

⚠ Needs revision

Scenario

A Sev-1 has been running 25 minutes. An engineer says they've found the bug and a fix is '10 minutes away'. Rolling back is available and would restore service in under 2 minutes but means losing an hour of legitimate feature work that shipped alongside the bug.

Leadership approach

Default bias: roll back. A known, fast path back to a known-good state almost always beats an unverified fix under time pressure — 'almost ready' fixes discovered mid-incident have a bad track record of taking longer than estimated or introducing a second issue.

The real question to ask the engineer: 'if this fix doesn't work in 10 minutes, are we in a worse position than we are now?' If yes, roll back immediately and let them finish the fix calmly, off the incident clock, then ship it properly.

Explicitly separate restoring service (the incident's job) from shipping the correct permanent fix (a follow-up task) — conflating the two is what causes bad in-incident decisions.

What this is really testing

Whether you can make a fast, calm, reversible decision under pressure and resist the natural pull to let a confident engineer 'just finish it' — commit to restoring service first, always.

✓ Confident

⚠ Needs revision

Scenario

Team A raised a connection pool timeout to reduce noisy errors. Team B, unrelated, increased their service's retry count for resilience. Together, under load, this created a retry storm that took down a shared dependency. Neither team did anything wrong in isolation.

Leadership approach

Explicitly state in the postmortem, and to both teams, that neither change was a mistake — this is a systemic visibility gap, not an individual or team failure. How you frame this in the room matters as much as the technical fix.

The actual gap: no shared visibility into changes across teams that touch a common dependency — surface this as the finding, not 'Team B should have known about Team A's change'.

Concrete action: a change calendar/notification for changes affecting shared/critical dependencies, and load testing that includes realistic multi-service retry behavior, not just single-service changes in isolation.

What this is really testing

Blameless facilitation in a scenario specifically designed to tempt blame — and whether you can extract a systemic, org-level fix rather than a technical band-aid on just one team's change.

✓ Confident

⚠ Needs revision

Scenario

A planned capacity increase was sized against last year's peak plus 50% headroom. Actual traffic hit 3x last year's peak. The platform degrades badly for 40 minutes before emergency scaling catches up.

Leadership approach

In the moment: emergency scale everything possible immediately (don't wait for root-cause clarity — the cause is obviously 'more load than capacity'), shed non-critical load/features to protect the core transaction path, and communicate honestly that this is a capacity event, not a code bug — that framing changes what stakeholders need to hear.

Follow-up, the harder part: the postmortem needs to interrogate why the estimate was 3x off — was it a flawed growth-rate assumption, a marketing push that wasn't communicated to engineering, or last year's data itself being an undercount? Fixing the number for next time without fixing the estimation process just repeats the miss with a

different multiplier.

Push for a standing practice: capacity planning becomes a cross-functional exercise (marketing/product share campaign plans with engineering ahead of time) and load testing validates the plan against a stress multiple beyond the estimate, not just the estimate itself.

What this is really testing

Whether you can separate the in-the-moment operational response from the deeper organizational/process fix — and whether you push accountability toward the process gap rather than a scapegoat.

✓ Confident

⚠ Needs revision

Scenario

The agreed error-budget policy says budget exhaustion triggers a feature freeze in favor of reliability work. A VP is pushing hard for an exception because the feature is tied to a customer commitment.

Leadership approach

Don't relitigate the policy under pressure — the entire point of agreeing an error-budget policy in advance is so this exact conversation doesn't become a political negotiation in the moment. Reiterate what was agreed and why.

But engage with the real tension, don't just say no: understand what's actually driving the customer commitment, and look for a middle path — can the feature ship behind a flag to a small cohort (limiting blast radius) while the reliability work continues, rather than a full freeze-or-ship binary?

Make the trade-off explicit and visible to leadership rather than making the call unilaterally in either direction: 'here's the reliability risk if we ship now, here's the customer/business risk if we don't — I recommend X, but this is a joint call given the stakes.'

Whatever's decided, feed it back into the policy — if this exception reveals the original policy was too rigid (or that it's working exactly as intended), that's useful data for the next quarter's agreement.

What this is really testing

Whether you'll hold a reliability line under real business pressure without being dogmatic about it — judgment, not rule-following, but judgment anchored in a pre-agreed framework rather than invented on the spot.

✓ Confident

⚠ Needs revision

Scenario

A production outage is actively being worked. Fifteen minutes in, security reports anomalous login patterns against the same service just before the outage began — possibly related, possibly coincidental.

Leadership approach

Don't assume causation, but don't dismiss it either — treat 'possibly a security incident' as true until ruled out, because the response posture differs: if this is an active attack, actions like restarting services or rolling back could destroy evidence or fail to address an attacker with continued access.

Bring security into the same incident structure immediately, not a parallel, disconnected process — one incident commander, security lead as a first-class participant with their own workstream, not a spectator.

Explicitly ask the question out loud in the bridge: 'if this is an active intrusion, does our planned fix (restart/rollback) help the attacker or destroy evidence we need?' — this single question is what a lot of engineers under pressure forget to ask.

If confirmed as security-related, this likely escalates to a formal security incident process (which in a bank has its own regulatory notification obligations) running alongside — not instead of — the operational recovery.

What this is really testing

Whether you default to 'move fast and fix it' reflexes even when a security angle changes what 'fixing it' safely means — recognizing when to slow down and loop in a different kind of expertise mid-incident.

✓ Confident

⚠ Needs revision